

EPGL assistant — API call flows

Assistant	EPGL — Emirates Post Group Licensing assistant (chat widget on epgl.ae)
Salesforce org	<code>https://epro--preprod2.sandbox.my.salesforce.com</code> (pre-prod sandbox)
REST API version	<code>v62.0</code> for reads, <code>v66.0</code> for composite sub-request URLs
Date	2 September 2026

Every call the assistant makes, in the order it makes them, with the payload. Each step says what the customer is doing, which call it triggers, and what we take from the response.

Auth. One OAuth 2.0 **client-credentials** grant against `/services/oauth2/token`, cached ~20 minutes and re-minted once on a 401. Salesforce rejects a `scope` parameter on this grant, so we do not send one. Every call below carries `Authorization: Bearer <that token>`.

Two kinds of call. **Reads** are SOQL over the standard query endpoint, with the statements fixed in our code — the model supplies a value, never the query. **Writes** are your Apex REST resources. Italics in amber mark an open question for your team.

Flow A — Identify the customer and their company

A1 · Customer signs in with UAE PASS

```
GET https://id.uaepass.ae/idshub/authorize?client_id=epg_web_prod&...
POST https://id.uaepass.ae/idshub/token
GET https://id.uaepass.ae/idshub/userinfo
```

Why Establish who they are. Not Salesforce — UAE PASS directly. **Take** the Emirates ID from `userinfo`. That is all we get, and it is the only key we can use to find their company.

A2 · Find their company from that Emirates ID

```
GET /services/data/v62.0/query?q=<SOQL>
```

```
SELECT Id, Name, EPG_Company_Name_Arabic__c, EPG_Trade_Name_in_English__c,
       EPG_Trade_Name_in_Arabic__c, EPG_Trade_license_no__c,
       EPG_Trade_license_Expiry_date__c, EPG_Emirates__c, EPG_Regulator__c,
       EPG_License_Number__c, EPG_License__c, EPG_License_Status__c,
       EPG_License_Expiry_Date__c,
       (SELECT FirstName, LastName, Email, Phone, EPG_Emirates_Id__c,
          LegalEntity_Profile_Person_EmiratesID__c, EPG_Designation__c
        FROM Contacts)
FROM Account
WHERE Id IN (SELECT AccountId FROM Contact
             WHERE EPG_Emirates_Id__c IN ('784199983926421',
                                           '784-1999-8392642-1'))
```

Why Sign-in gives an Emirates ID, not a trade licence number, so the company has to be reached through Contact. **Both spellings are queried** — bare digits and dashed — because a record stored one way would

otherwise read as "no company". **Take** Id (the Account id), the trade licence number, EPG_License__c (the licence RECORD id — needed later), and the contacts.

Note A person can be a contact on several companies, so this returns a list and the customer chooses. We never assume the first.

Open: is there a supported way to resolve an Account from an Emirates ID directly? We were told the AccountByLicense Apex resource applies the ownership rule, but our integration user is not granted that class (403), so we join through Contact ourselves.

A3 · Or find it from a trade licence number

```
SELECT <same fields> FROM Account WHERE EPG_Trade_license_no__c = '1234567'
```

Why The fallback when sign-in produces no match, or for a guest.

A3b · Or by Account id, once known

```
SELECT <same fields> FROM Account WHERE Id = '0015f00000XXXXXXXX'
```

A4 · Show their quarterly history

```
SELECT Id, Name, EPG_Year__c, Quarter__c, EPG_Status__c, EPG_Approval_Status__c,
       EPG_Submitted_Date__c, EPG_License__c, EPG_License__r.Name,
       EPG_License_Start_Date__c, EPG_License_End_Date__c,
       EPG_Postal_License_Number__c, EPG_Total_Revenue__c,
       EPG_Total_Revenue_for_Leviable_Services__c,
       Total_Revenue_for_Non_Leviable_Services__c,
       EPG_Calculated_Levy_Amount__c, EPG_Due_Fees_for_the_period__c,
       EPG_Amount_Paid__c, Payable_Balance_For_Customer__c
FROM EPG_Form_9__c
WHERE EPG_Company_Name__c = '0015f00000XXXXXXXX'
ORDER BY EPG_Year__c DESC, Quarter__c DESC
LIMIT 12
```

Why Form 9 is what IDEP files each quarter, so these are the quarterly figures we pre-fill a renewal from — the customer does not retype them. **Take** EPG_License__c in particular: it is the licence record id the renewal finance rows require, and your example query returns the licence *name* instead.

Flow B — Apply for a new licence

B1 · Check it is not a duplicate

POST /services/apexrest/EPGL/LicenseRequest/duplicate-check

Why Before any brand-new submission, so we do not create a second application for something already in flight.

Take `matchFound`, `matchedRecordIds`, `matchedRequests []` with each one's status, number and `recommendedAction`.

Note: this does not return an Account id, so it cannot be used to find one.

B2 · Submit the application — one atomic composite

POST /services/apexrest/EPGL/LicenseRequest

```

{ "allOrNone": true,
  "isAgentSource": true,
  "compositeRequest": [
    { "method": "POST", "referenceId": "NewAccount",
      "url": "/services/data/v66.0/subjects/Account",
      "body": { "Name": "Example Courier LLC",
        "EPG_Company_Name_Arabic__c": "...",
        "EPG_Trade_license_no__c": "1234567",          // the upsert key
        "EPG_Trade_license_Expiry_date__c": "2027-05-30",
        "EPG_Emirates__c": "Dubai",
        "RecordTypeId": "0125f000001xIheAAE" } },

    { "method": "POST", "referenceId": "NewPartner",
      "url": "/services/data/v66.0/subjects/EPG_Partner__c",
      "body": { "EPG_Account__c": "@{NewAccount.id}", ... } },

    { "method": "POST", "referenceId": "NewContact",
      "url": "/services/data/v66.0/subjects/Contact",
      "body": { "AccountId": "@{NewAccount.id}",
        "EPG_Emirates_Id__c": "784...", "EPG_Designation__c": "Manager", ... } },

    { "method": "POST", "referenceId": "NewUser",
      "url": "/services/data/v66.0/subjects/User",
      "body": { "EPG_Emirates_Id__c": "784...", ... } },

    { "method": "POST", "referenceId": "NewMember",
      "url": "/services/data/v66.0/subjects/Members__c", "body": { ... } },

    { "method": "POST", "referenceId": "NewDocument",
      "url": "/services/data/v66.0/subjects/EPG_Document__c", "body": { ... } },

    { "method": "POST", "referenceId": "NewLicenseRequest",
      "url": "/services/data/v66.0/subjects/EPG_License_Request__c",
      "body": { "EPG_Account__c": "@{NewAccount.id}",
        "RecordTypeId": "0125f000001xIhuAAE",
        "EPG_Service__c": "a1H5f0000033Q7pEAE",
        "serviceId": "S-EPG-000002",
        "serviceNameEN": "Issue Postal Activity License",
        "Terms_Conditions_Accepted__c": true } } ] }

```

What we learned the hard way, all of it worth confirming with you

HTTP is always 200.	Success is judged only by each item's <code>body.success</code> . A failed submission still returns 200.
Every item needs <code>method</code> , <code>referenceId</code> and a <code>url</code> .	A sub-request without a valid <code>url</code> fails the whole submit with <code>Invalid</code> or <code>missing URL</code> .
The Account is matched by trade licence number, not by Id.	Send the Account without an <code>Id</code> and with <code>EPG_Trade_license_no__c</code> ; the handler updates the existing record. A "duplicate / unique conflict" on the Account means the company already exists — resend the same licence number, do not create a second company.
Array-body items fan out with <code><index></code> .	<code>NewContact</code> becomes <code>NewContact_0</code> , <code>NewContact_1</code> ... in the response.
<code>EPG_Document__c</code> items are processed internally and omitted from the response.	So their absence is not a failure.
On any failure every item echoes the same message.	Rolled back due to <code>allOrNone=true</code> : ... — we relay the underlying error rather than that wrapper.
Account fields are nearly all read-only to the portal.	Editable: <code>Name</code> , <code>EPG_Trade_license_no__c</code> , <code>RecordTypeId</code> , <code>EPG_Trade_license_Expiry_date__c</code> , <code>EPG_Trade_Name_in_English__c</code> , <code>EPG_Trade_Name_in_Arabic__c</code> . The English company name is the standard <code>Name</code> field — there is no English-name custom field.

B3 · Attach the documents

POST `/services/apexrest/EPGL/Document`

Base64, linked to the `NewLicenseRequest` id. Trade licence, MOA, Emirates ID.

Note the Declaration & Undertaking is captured in the conversation, not as a file:

`Terms_Conditions_Accepted__c: true` on the licence request, with the acceptance timestamp held on our side. We never ask for a signed PDF of it.

B4 · Report status back to the customer

GET `/services/apexrest/EPGL/LicenseRequest/status?id=<NewLicenseRequest id>`

Flow C — Renew a licence

Flow B's shape, with a different record type and four extra requirements.

C1 · Read the company and its licence first

Flow A2 or A3, then A4. This is not optional for a renewal — two values only exist there:

- `accountId` → the renewal Account item **must** carry its `Id`. Without it the handler attempts an insert and fails on the Account's unique field.
- `EPG_License__c` (the licence **record id**) → every finance row needs it.

C2 · Submit the renewal

POST `/services/apexrest/EPGL/LicenseRequest`

```
{ "allOrNone": true, "isAgentSource": true,
  "compositeRequest": [
    { "method": "POST", "referenceId": "NewAccount",
      "url": "/services/data/v66.0/subjects/Account",
      "body": { "Id": "0015f00000XXXXXX",           // required on a renewal
                "EPG_Trade_license_no__c": "1234567",
                "EPG_Trade_license_Expiry_date__c": "2027-05-30",
                "RecordTypeId": "0125f000001xIheAAE" } },

    { "method": "POST", "referenceId": "NewContact",
      "url": "/services/data/v66.0/subjects/Contact",
      "body": { "AccountId": "@{NewAccount.id}",
                "EPG_Designation__c": "Accountant", ... } },           // mandatory

    { "method": "POST", "referenceId": "NewLicenseRequest",
      "url": "/services/data/v66.0/subjects/EPG_License_Request__c",
      "body": { "EPG_Account__c": "@{NewAccount.id}",
                "RecordTypeId": "0125f000001xIhwAAE",
                "EPG_Service__c": "a1H5f0000033Q7lEAE",
                "serviceId": "S-EPG-000003",
                "serviceNameEN": "Renew Postal Activity License",
                "EPG_Terms_and_Conditions__c": true,
                "Approved_Commitment_Form__c": true,
                "Mandatory_integration_with_IDEP__c": true,
                "EPG_Is_Financial_Statement_Submitted__c": true } },

    { "method": "POST", "referenceId": "Finance1",
      "url": "/services/data/v66.0/subjects/EPG_Finance_Summary__c",
      "body": { "EPG_License_Request__c": "@{NewLicenseRequest.id}",
                "EPG_License_No__c": "a0X5f00000YYYYYYYY",           // licence RECORD id
                "Quarter__c": "Q3", "EPG_Year__c": "2023",
                "Name": "Q3 2023",
                "EPG_Leviable_Income__c": 250000,
                "EPG_Non_Leviable_Income__c": 0 } },

    /* ...one per quarter, walking forward from the licence period start... */ ] }
```

Four renewal-specific things

All four flags must be true.	EPG_Terms_and_Conditions__c, Approved_Commitment_Form__c, Mandatory_integration_with_IDEP__c, EPG_Is_Financial_Statement_Submitted__c. We only set them from an explicit in-chat confirmation.
EPG_License_No__c is an ID-type lookup.	It takes the licence record id, not the printed licence number. Sending the number fails the whole composite with License No: id value of incorrect type.
EPG_Finance_Summary__c has no upsert key.	So a second submit for the same renewal creates duplicate quarters. We check getRequestStatus before retrying, ever.
Quarters are real calendar quarters.	Walked forward from the licence period start, rolling the year at Q4 — start Q3 2023 gives Q3 2023, Q4 2023, Q1 2024, Q2 2024.

Expected responses

- The backend auto-sets Under document review on renewals — we tell the customer that rather than treating it as pending action.
- No License found for this Account means there is nothing to renew; we offer a new application instead.

Flow D — Pay a licence fee

The customer pays on our gateway. Salesforce creates the Payment Advice, Invoice and Receipt itself — your team was explicit that we create none of those. What Salesforce cannot know is that the money arrived, so we tell it:

```
POST /services/apexrest/paymentNotification/
```

```
{ "entityId": "Emirates Post Group Licensing – AI Assistant",
  "notifyPayment": {
    "salesforceId": "a0Y5f00000ZZZZZZZ",           // EPG_License_Request__c id
    "payment": {
      "paymentId": "PAY-2026-0001",
      "payOn": "2026-09-02T08:14:00.000Z",
      "payThru": "N-Genius",
      "paymentMethod": "CreditCard",
      "paymentStatus": "completed",
      "transactions": [
        { "transactionId": "...", "amount": 5000,
          "currency": { "en": "AED", "ar": "درهم" } } ] } } }
```

Fired from our payment webhook once the gateway settles, never from the model.

Two wire quirks from your own spec, noted so nobody tidies them away

- `currency` and `desc` are reserved words in Apex and are remapped internally, so those exact keys must appear on the wire.
- The `400` body still names a legacy `applicationId` element; the field it means is `notifyPayment.salesforceId`.

On success Salesforce creates the `Payment_Notification__c` against the request's Payment Advice, sets the request to Payment Verified, marks the Advice Paid with the summed transaction amount, and approves the Payment Items.

What we deliberately do NOT do

The model never writes SOQL.	Every query above is a fixed template in our code; the model supplies one value, which is shape-checked first. A licence number must match <code>^[A-Za-z0-9] [A-Za-z0-9\-\ /]{0,38}\$</code> , an Emirates ID must be 15 digits, an id 15–18 alphanumerics. The <code>q</code> parameter is not a tool input, so an injected instruction in an uploaded document cannot read the org.
We create no Payment Advice, Invoice or Receipt.	Only the notification.
We never submit a renewal twice.	<code>EPG_Finance_Summary__c</code> has no upsert key.
We never ask for a signed declaration document.	It is a checkbox in the conversation, mapped to the boolean fields.

Open questions

1. `AccountByLicense` — our integration user gets a 403 on that Apex class, so we join Account → Contact ourselves to resolve a company from an Emirates ID. Should we be granted it, or is the join the intended route?
2. **An Account id from** `duplicate-check`. It returns `matchedRecordIds` and `matchedRequests`, but nothing that identifies the Account, so a renewal has to reach the id through the SOQL read. Is that intended?
3. `EPG_Finance_Summary__c` **upsert key**. Without one, any retry duplicates quarters. Could `EPG_License_Request__c` + `Quarter__c` + `EPG_Year__c` be made an external id?
4. **HTTP 200 on failure**. Judging by `body.success` per item works, but a non-2xx on a rolled-back composite would remove a whole class of mistake.
5. **Production org**. Everything above runs against `epro--preprod2.sandbox`. We have no production instance URL or credentials, so the production assistant is still pointed at the sandbox.
6. **Field-level access**. Confirmation of exactly which Account fields the portal integration user may write. We work from six; more would remove some re-asking.